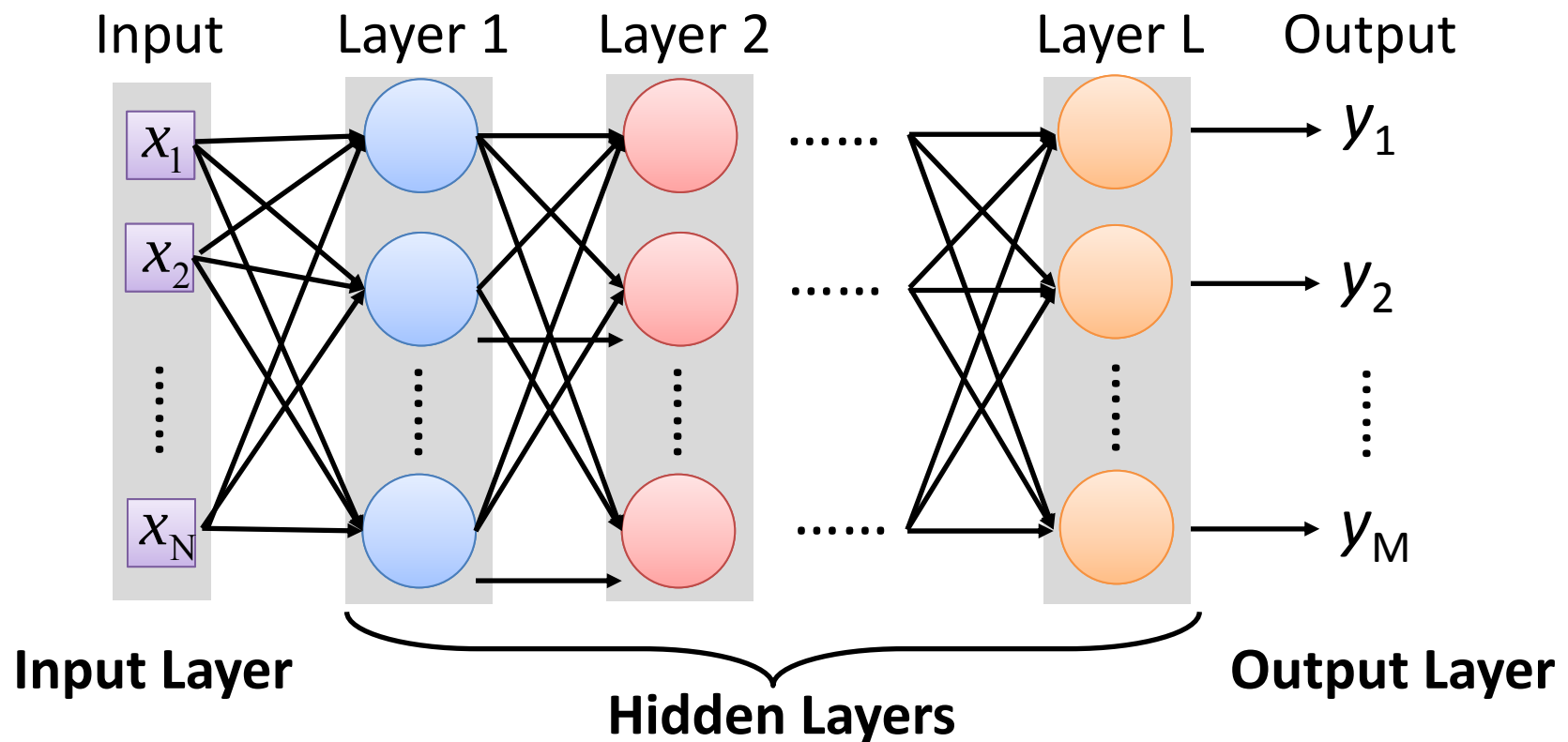


# Elements of Neural Networks

## *Introduction to Neural Networks*

- Deep NNs have many hidden layers
  - **Fully-connected** (**dense**) layers (a.k.a. **Multi-Layer Perceptron** or MLP)
  - Each neuron is connected to all neurons in the succeeding layer

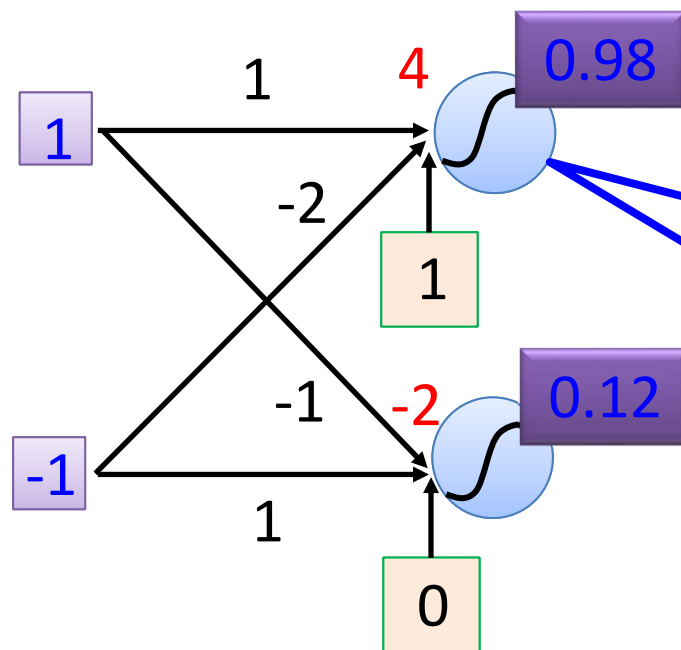


# Elements of Neural Networks

## Introduction to Neural Networks

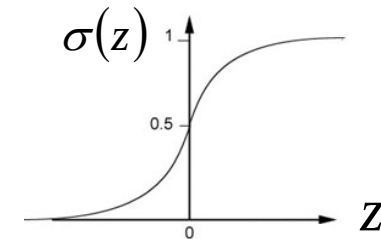
- A simple network, toy example

$$(1 \cdot 1) + (-1) \cdot (-2) + 1 = 4$$



Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

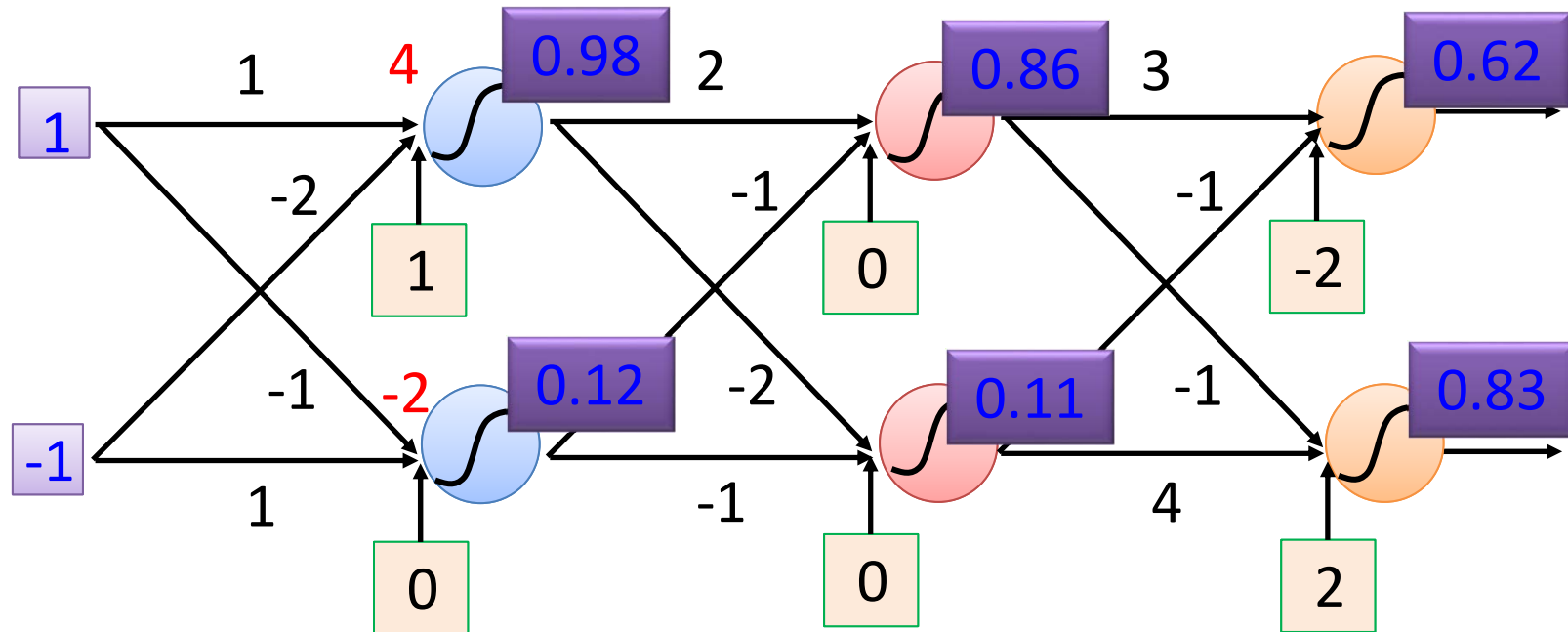


$$1 \cdot (-1) + (-1) \cdot 1 + 0 = -2$$

# Elements of Neural Networks

## Introduction to Neural Networks

- A simple network, toy example (cont'd)
  - For an input vector  $[1 \ -1]^T$ , the output is  $[0.62 \ 0.83]^T$

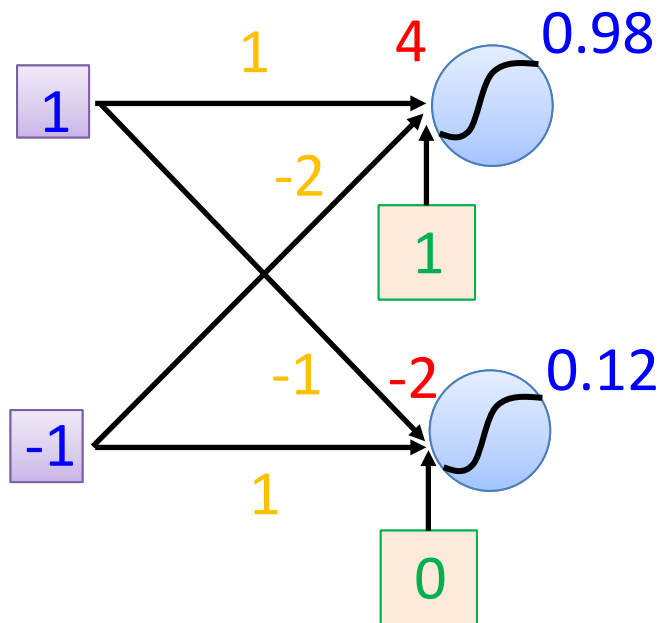


$$f: R^2 \rightarrow R^2 \quad f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix}$$

# Matrix Operation

*Introduction to Neural Networks*

- Matrix operations are helpful when working with multidimensional inputs and outputs



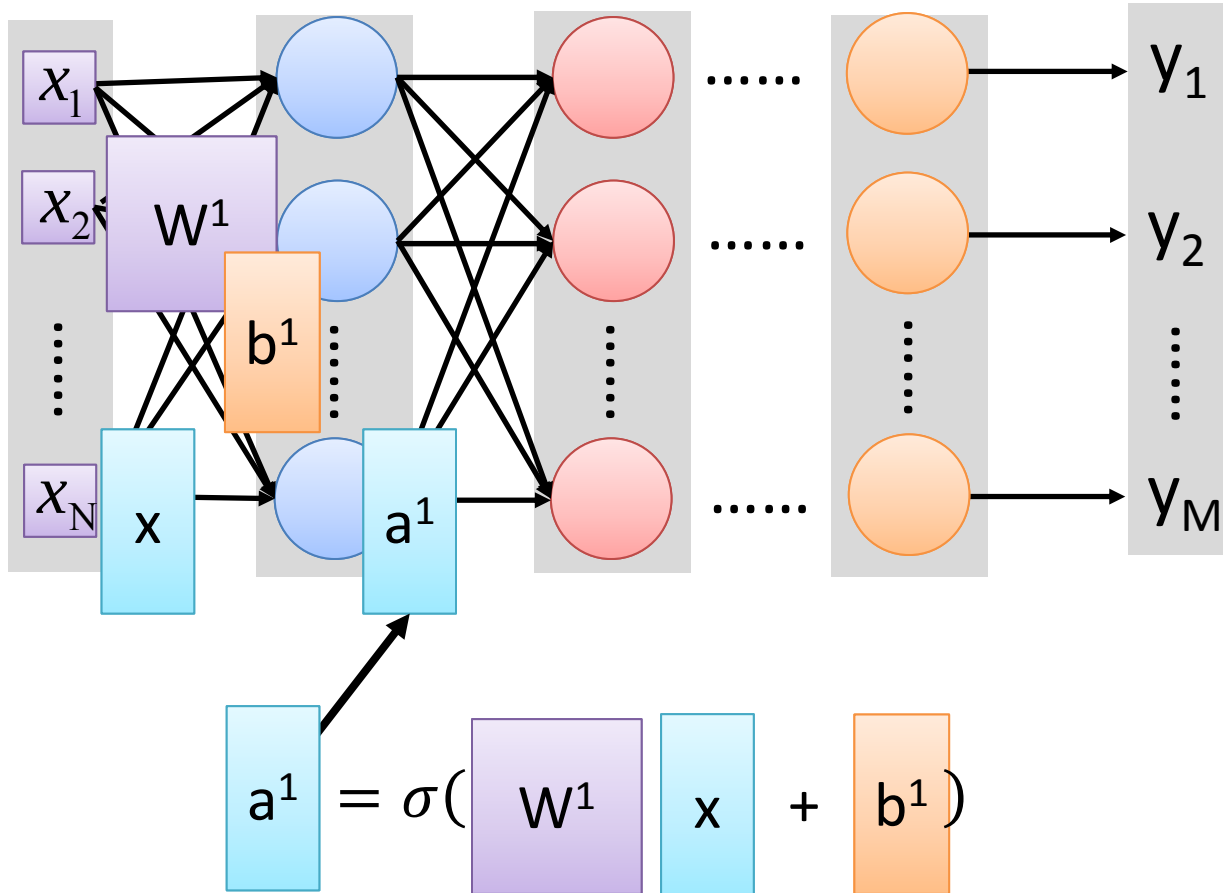
$$\sigma( W x + b ) = a$$

$$\sigma( \underbrace{\begin{bmatrix} 1 & -2 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix}}_{\begin{bmatrix} 4 \\ -2 \end{bmatrix}} ) = \begin{bmatrix} 0.98 \\ 0.12 \end{bmatrix}$$

# Matrix Operation

## *Introduction to Neural Networks*

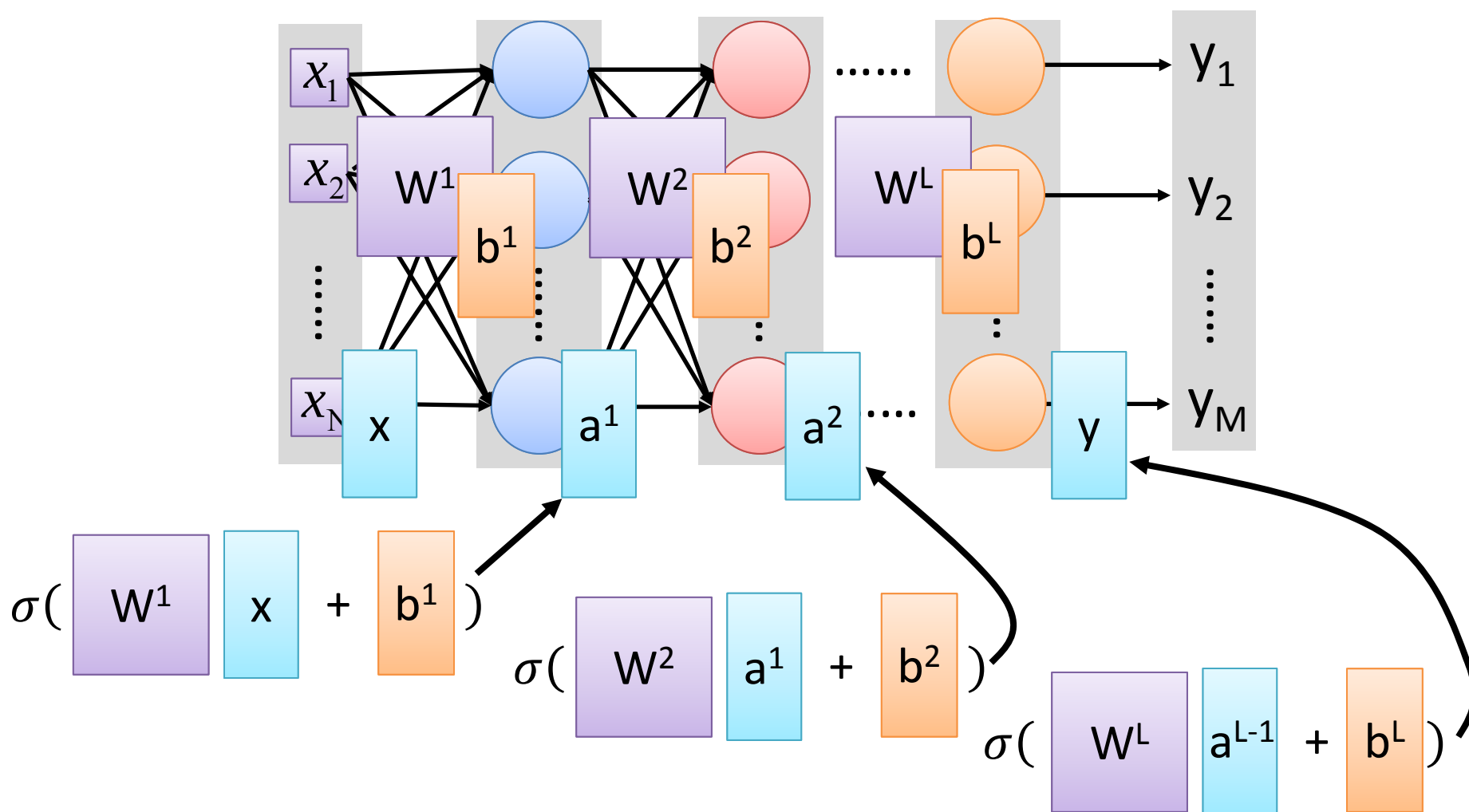
- Multilayer NN, matrix calculations for the first layer
  - Input vector  $x$ , weights matrix  $W^1$ , bias vector  $b^1$ , output vector  $a^1$



# Matrix Operation

*Introduction to Neural Networks*

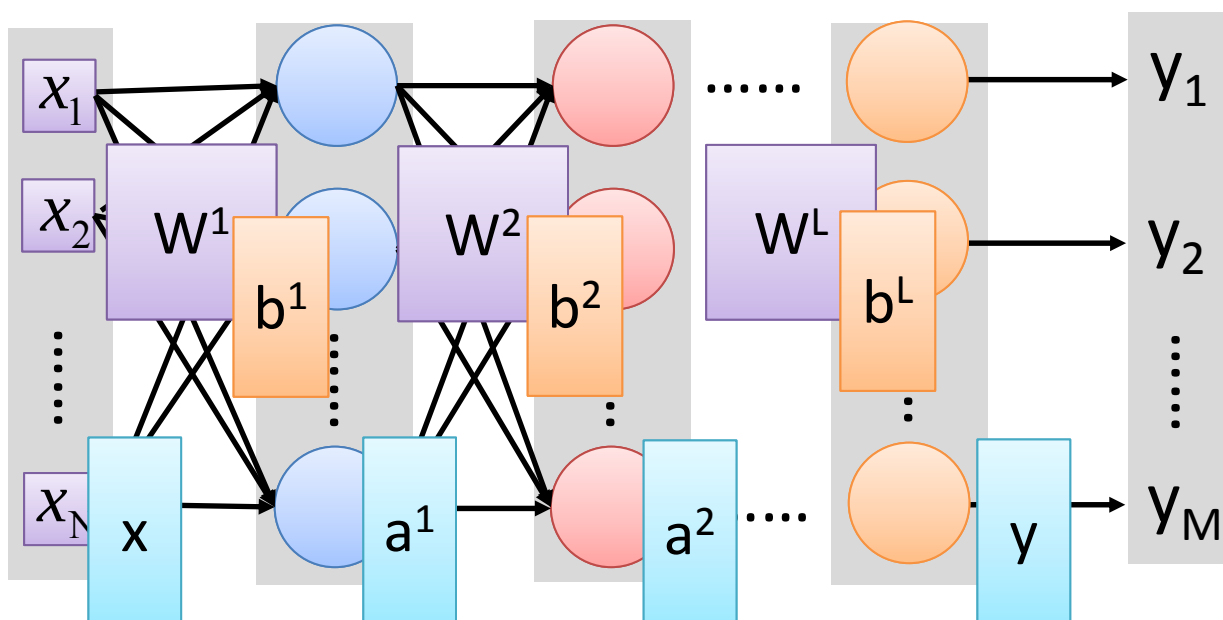
- Multilayer NN, matrix calculations for all layers



# Matrix Operation

*Introduction to Neural Networks*

- Multilayer NN, function  $f$  maps inputs  $x$  to outputs  $y$ , i.e.,  $y = f(x)$



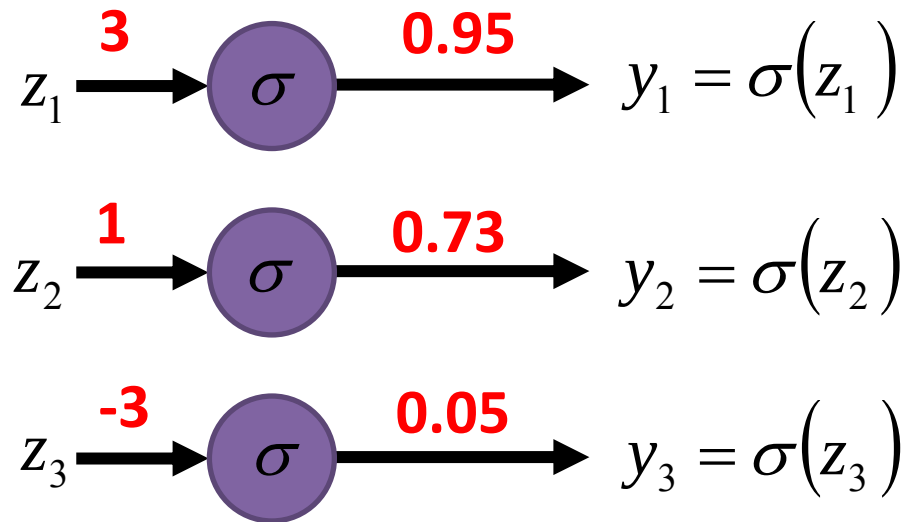
$$y = f(x) = \sigma(W^L \dots \sigma(W^2 \sigma(W^1 x + b^1) + b^2) \dots + b^L)$$

# Softmax Layer

## Introduction to Neural Networks

- In **multi-class classification** tasks, the output layer is typically a *softmax layer*
  - I.e., it employs a *softmax activation function*
  - If a layer with a sigmoid activation function is used as the output layer instead, the predictions by the NN may not be easy to interpret
    - Note that an output layer with sigmoid activations can still be used for binary classification

### A Layer with Sigmoid Activations





# Softmax Layer

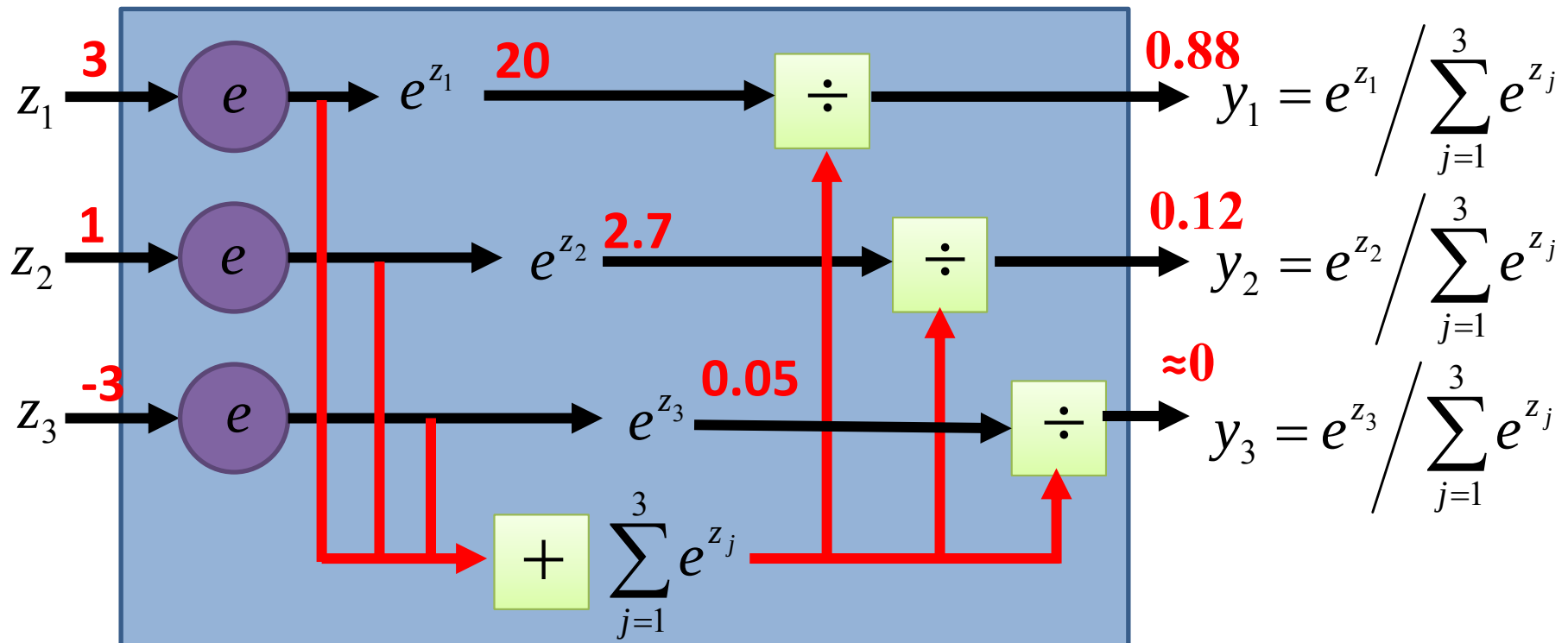
Introduction to Neural Networks

- The **softmax layer** applies softmax activations to output a probability value in the range  $[0, 1]$ 
  - The values  $z$  inputted to the softmax layer are referred to as *logits*

**Probability:**

- $0 < y_i < 1$
- $\sum_i y_i = 1$

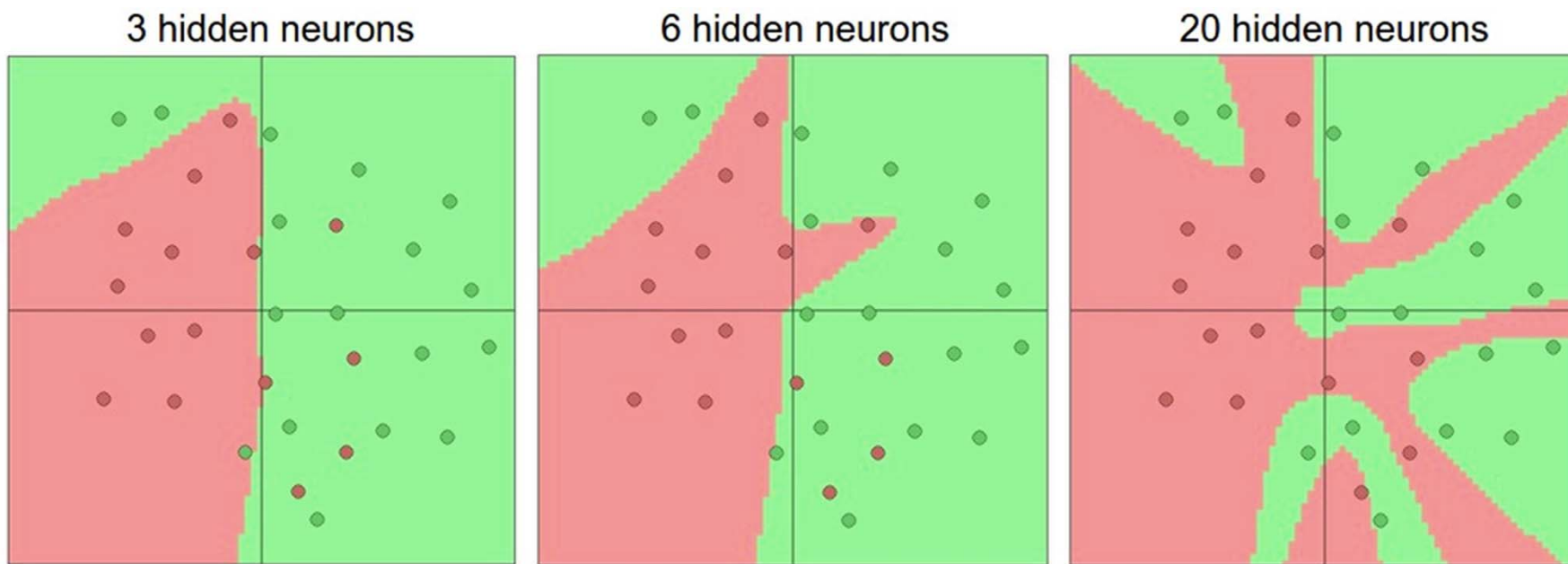
## A Softmax Layer



# Activation Functions

## *Introduction to Neural Networks*

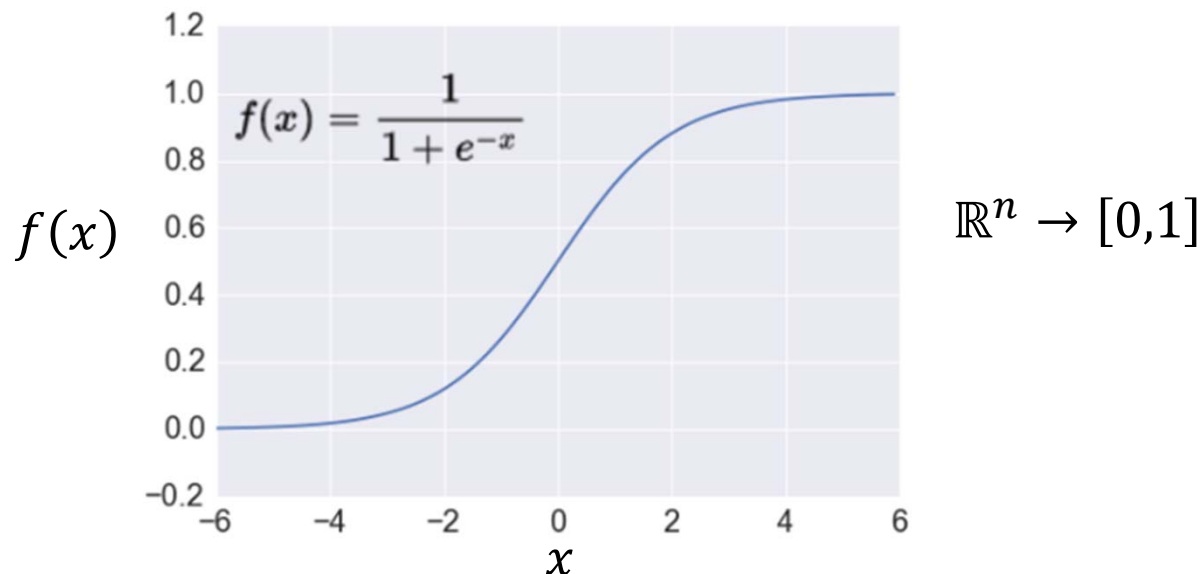
- **Non-linear activations** are needed to learn complex (non-linear) data representations
  - Otherwise, NNs would be just a linear function (such as  $W_1 W_2 x = Wx$ )
  - NNs with large number of layers (and neurons) can approximate more complex functions
    - Figure: more neurons improve representation (but, may overfit)



# Activation: Sigmoid

*Introduction to Neural Networks*

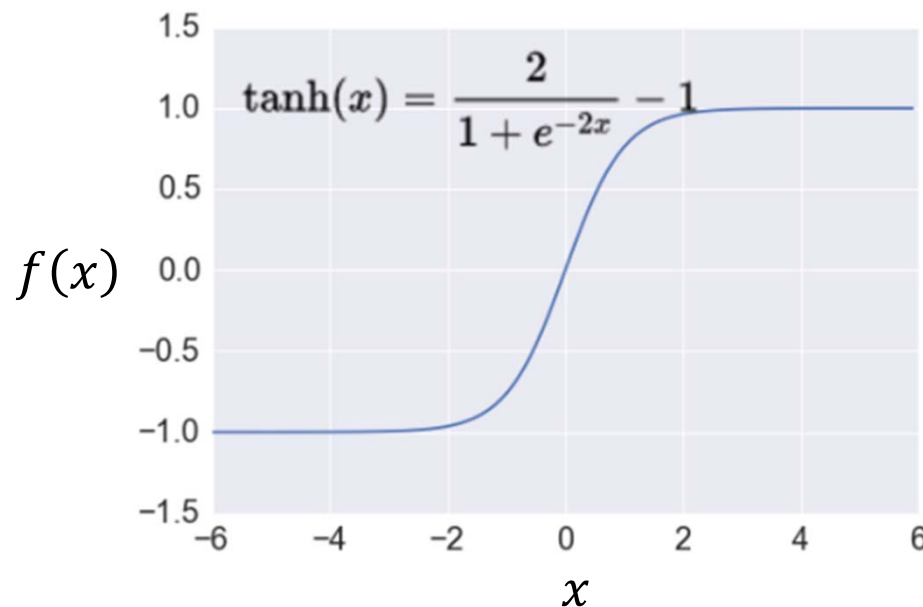
- **Sigmoid function**  $\sigma$ : takes a real-valued number and “squashes” it into the range between 0 and 1
  - The output can be interpreted as the firing rate of a biological neuron
    - Not firing = 0; Fully firing = 1
  - When the neuron’s activation are 0 or 1, sigmoid neurons saturate
    - Gradients at these regions are almost zero (almost no signal will flow)
  - Sigmoid activations are less common in modern NNs



# Activation: Tanh

*Introduction to Neural Networks*

- **Tanh function**: takes a real-valued number and “squashes” it into range between -1 and 1
  - Like sigmoid, tanh neurons saturate
  - Unlike sigmoid, the output is zero-centered
    - It is therefore preferred than sigmoid
  - Tanh is a scaled sigmoid:  $\tanh(x) = 2 \cdot \sigma(2x) - 1$



$$\mathbb{R}^n \rightarrow [-1,1]$$

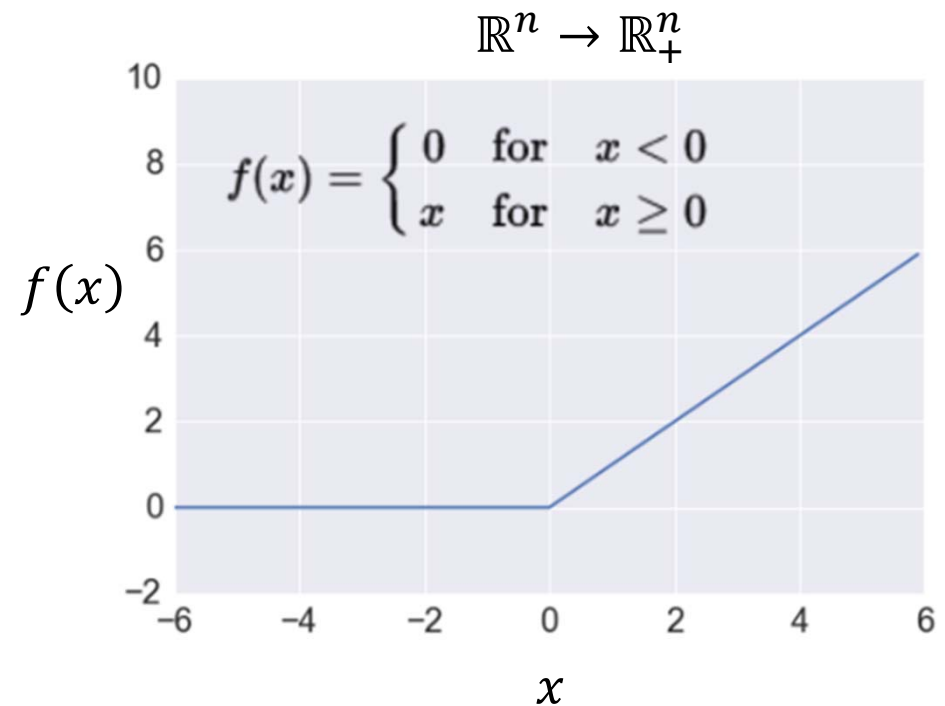
# Activation: ReLU

*Introduction to Neural Networks*

- **ReLU** (Rectified Linear Unit): takes a real-valued number and thresholds it at zero

$$f(x) = \max(0, x)$$

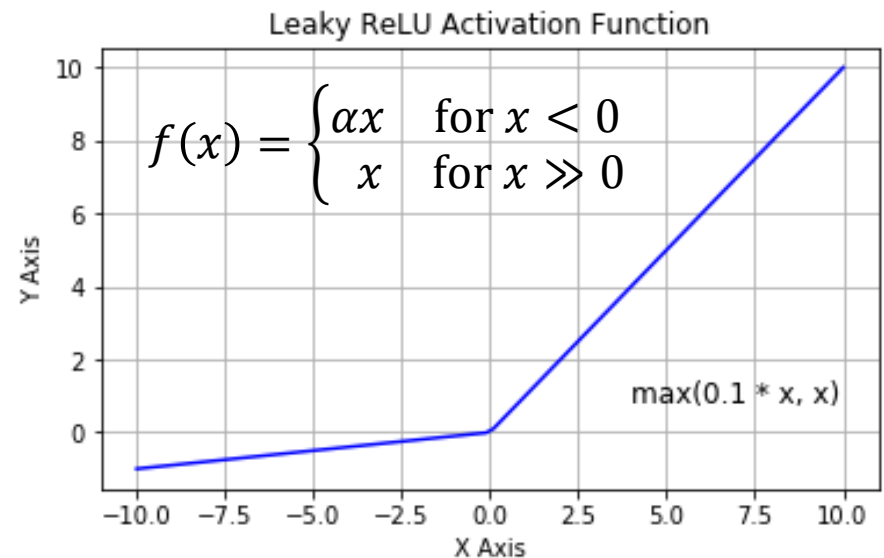
- Most modern deep NNs use ReLU activations
- ReLU is fast to compute
  - Compared to sigmoid, tanh
  - Simply threshold a matrix at zero
- Accelerates the convergence of gradient descent
  - Due to linear, non-saturating form
- Prevents the gradient vanishing problem



# Activation: Leaky ReLU

## Introduction to Neural Networks

- The problem of ReLU activations: they can “die”
  - ReLU could cause weights to update in a way that the gradients can become zero and the neuron will not activate again on any data
  - E.g., when a large learning rate is used
- **Leaky ReLU** activation function is a variant of ReLU
  - Instead of the function being 0 when  $x < 0$ , a leaky ReLU has a small negative slope (e.g.,  $\alpha = 0.01$ , or similar)
  - This resolves the dying ReLU problem
  - Most current works still use ReLU
    - With a proper setting of the learning rate, the problem of dying ReLU can be avoided

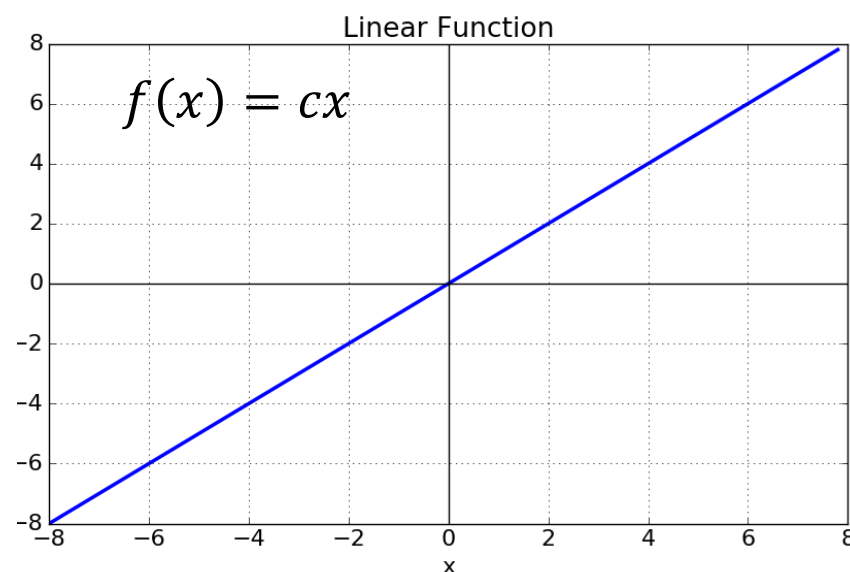


# Activation: Linear Function

*Introduction to Neural Networks*

- **Linear function** means that the output signal is proportional to the input signal to the neuron
  - If the value of the constant  $c$  is 1, it is also called **identity activation function**
  - This activation type is used in regression problems
    - E.g., the last layer can have linear activation function, in order to output a real number (and not a class membership)

$$\mathbb{R}^n \rightarrow \mathbb{R}^n$$



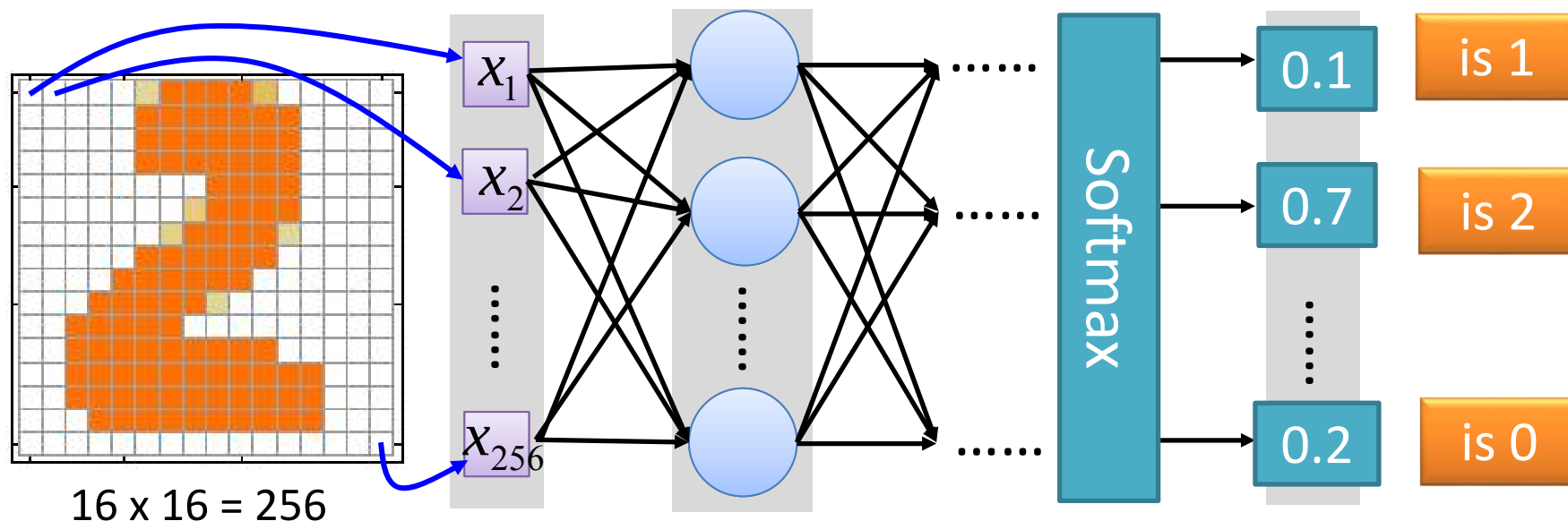
# Training NNs

## Training Neural Networks

- The network *parameters*  $\theta$  include the **weight matrices** and **bias vectors** from all layers

$$\theta = \{W^1, b^1, W^2, b^2, \dots, W^L, b^L\}$$

- Often, the model parameters  $\theta$  are referred to as **weights**
- Training a model to learn a set of parameters  $\theta$  that are optimal (according to a criterion) is one of the greatest challenges in ML

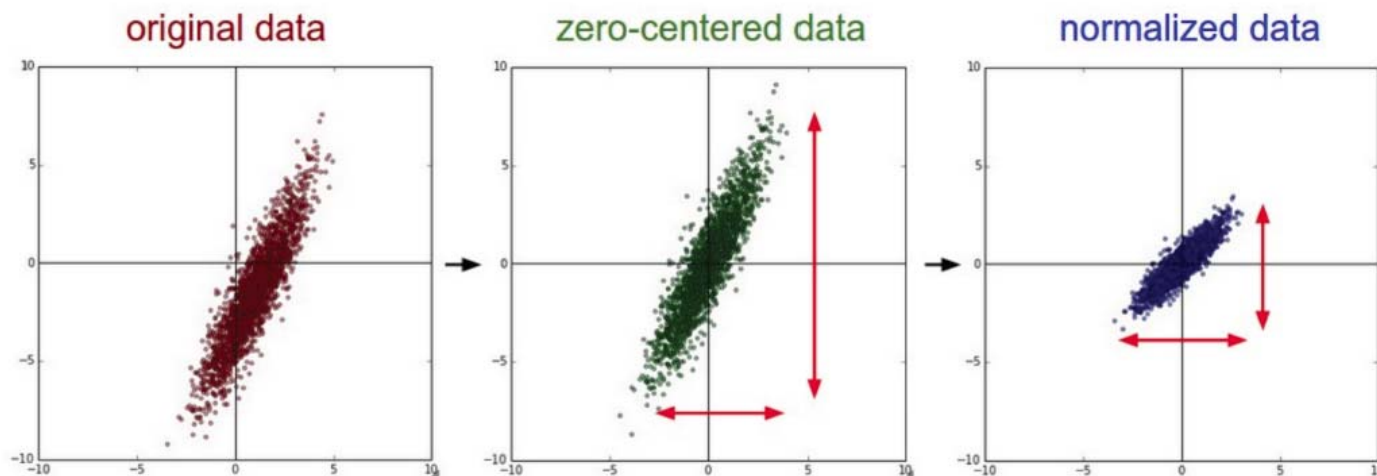




# Training NNs

## Training Neural Networks

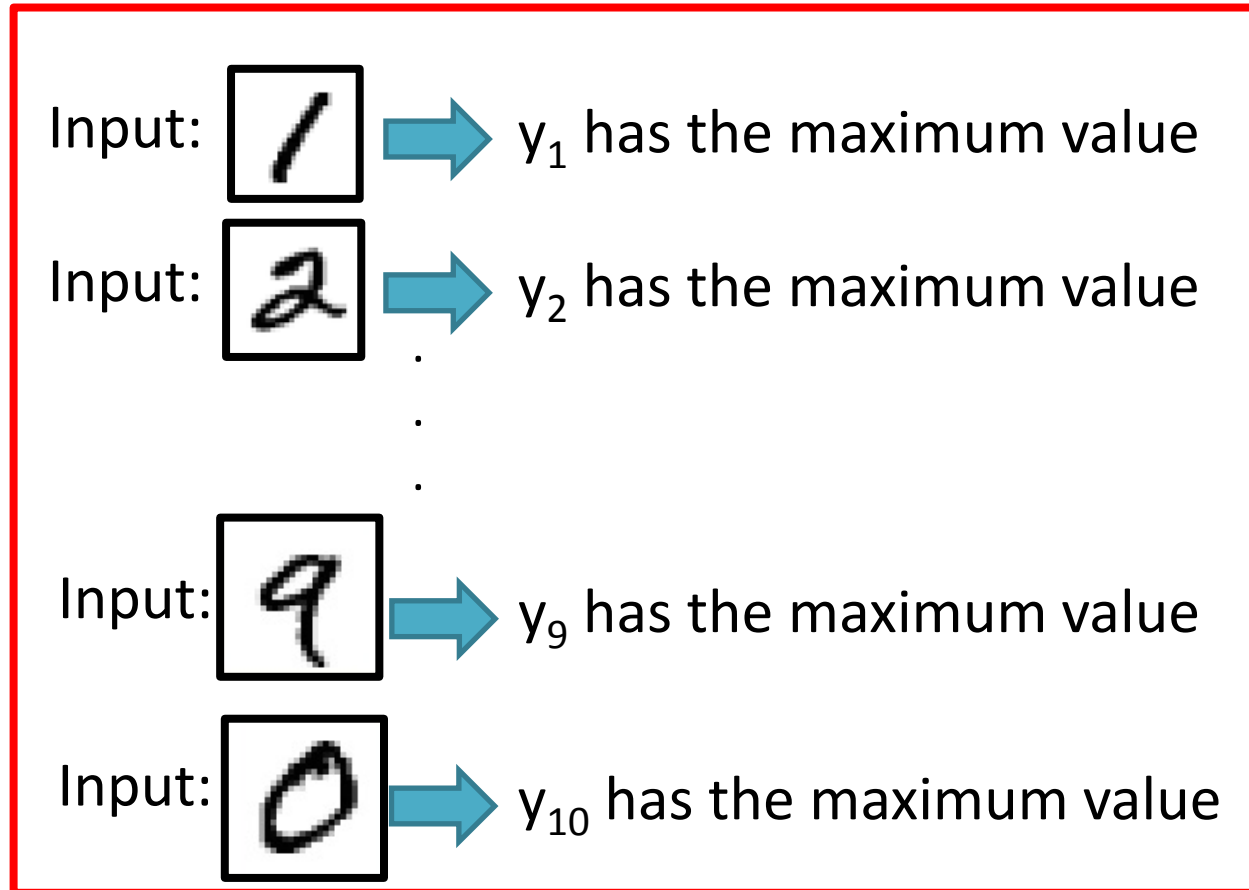
- **Data preprocessing** - helps convergence during training
  - **Mean subtraction**, to obtain zero-centered data
    - Subtract the mean for each individual data dimension (feature)
  - **Normalization**
    - Divide each feature by its standard deviation
      - To obtain standard deviation of 1 for each data dimension (feature)
    - Or, scale the data within the range  $[0,1]$  or  $[-1, 1]$ 
      - E.g., image pixel intensities are divided by 255 to be scaled in the  $[0,1]$  range



# Training NNs

## *Training Neural Networks*

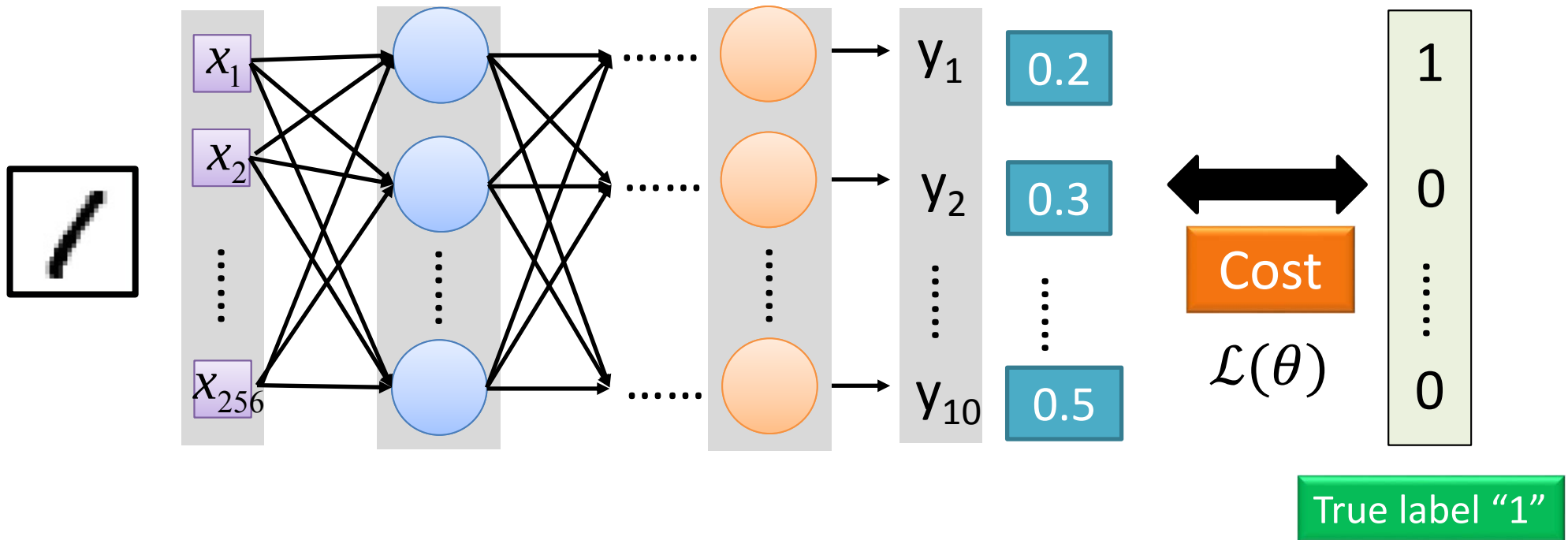
- To train a NN, set the parameters  $\theta$  such that for a training subset of images, the corresponding elements in the predicted output have maximum values



# Training NNs

## Training Neural Networks

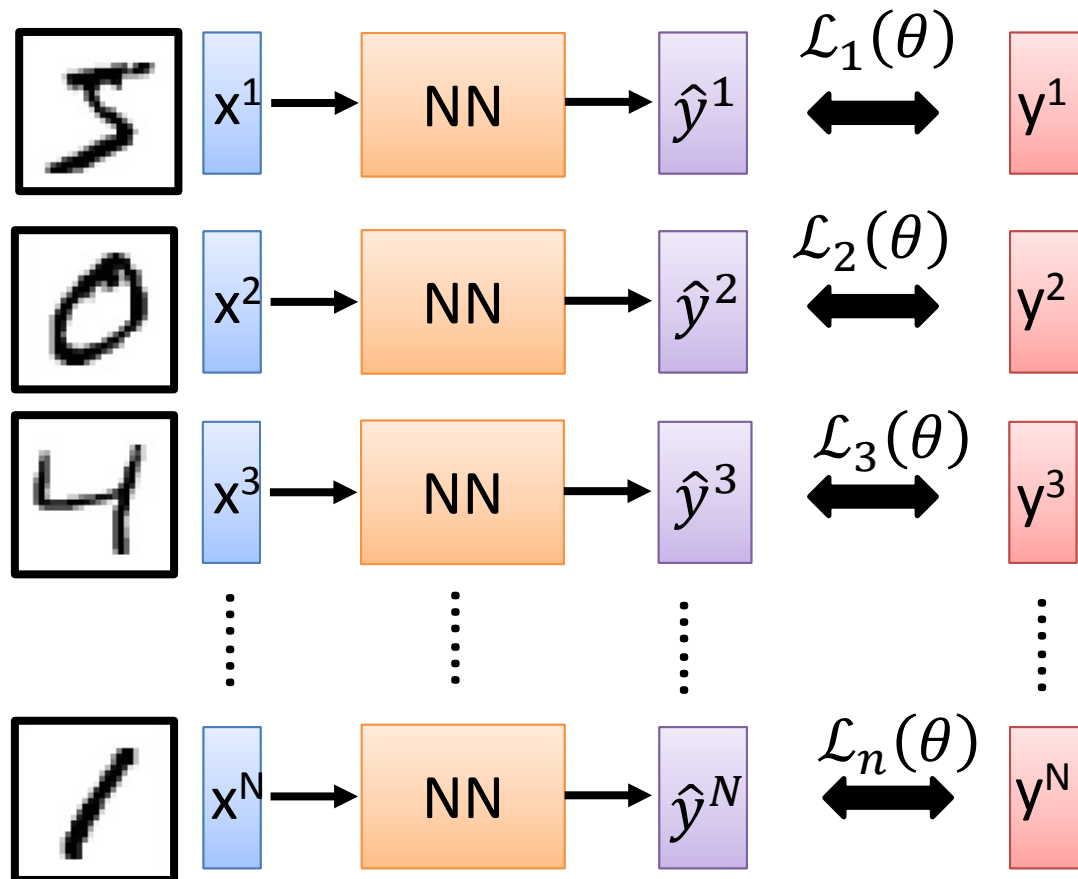
- Define a **loss function/objective function/cost function**  $\mathcal{L}(\theta)$  that calculates the difference (error) between the model prediction and the true label
  - E.g.,  $\mathcal{L}(\theta)$  can be mean-squared error, cross-entropy, etc.



# Training NNs

## Training Neural Networks

- For a training set of  $N$  images, calculate the total loss overall all images:  $\mathcal{L}(\theta) = \sum_{n=1}^N \mathcal{L}_n(\theta)$
- Find the optimal parameters  $\theta^*$  that minimize the total loss  $\mathcal{L}(\theta)$



# Loss Functions

*Training Neural Networks*

- *Classification tasks*

**Training  
examples**

Pairs of  $N$  inputs  $x_i$  and ground-truth class labels  $y_i$

**Output  
Layer**

Softmax Activations  
[maps to a probability distribution]

$$P(y = j \mid \mathbf{x}) = \frac{e^{\mathbf{x}^\top \mathbf{w}_j}}{\sum_{k=1}^K e^{\mathbf{x}^\top \mathbf{w}_k}}$$

**Loss function**

**Cross-entropy**  $\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \left[ y_k^{(i)} \log \hat{y}_k^{(i)} + (1 - y_k^{(i)}) \log (1 - \hat{y}_k^{(i)}) \right]$

Ground-truth class labels  $y_i$  and model predicted class labels  $\hat{y}_i$

# Loss Functions

*Training Neural Networks*

- *Regression tasks*

**Training  
examples**

Pairs of  $N$  inputs  $x_i$  and ground-truth output values  $y_i$

**Output  
Layer**

Linear (Identity) or Sigmoid Activation

**Loss function**

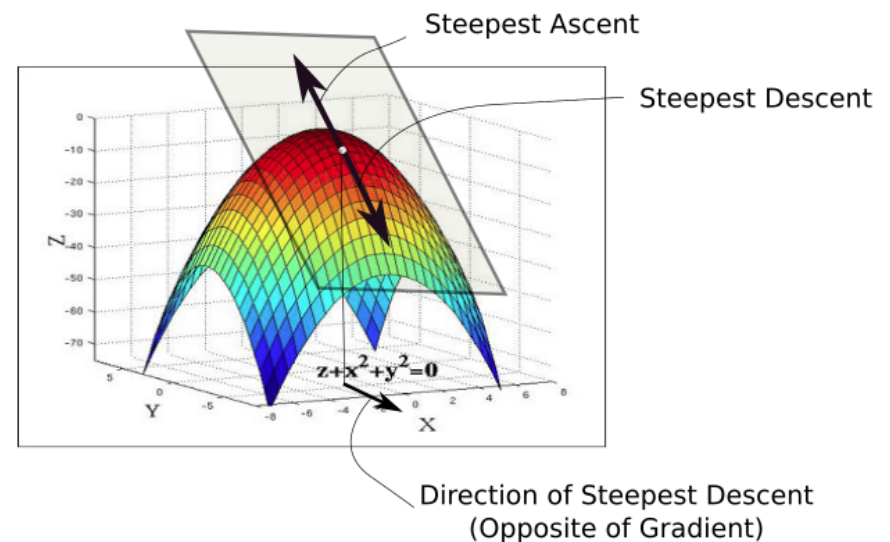
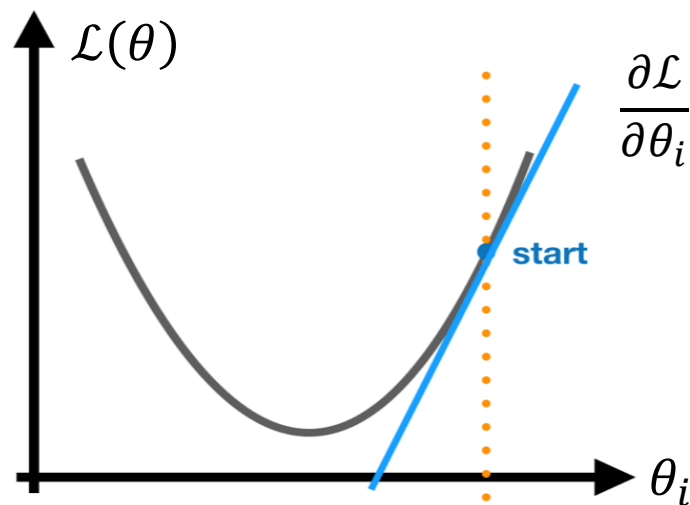
**Mean Squared Error**  $\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$

**Mean Absolute Error**  $\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n |y^{(i)} - \hat{y}^{(i)}|$

# Training NNs

## Training Neural Networks

- Optimizing the loss function  $\mathcal{L}(\theta)$ 
  - Almost all DL models these days are trained with a variant of the *gradient descent* (GD) algorithm
  - GD applies iterative refinement of the network **parameters  $\theta$**
  - GD uses the opposite direction of the **gradient** of the loss with respect to the NN parameters (i.e.,  $\nabla \mathcal{L}(\theta) = [\partial \mathcal{L} / \partial \theta_i]$ ) for updating  $\theta$ 
    - The gradient of the loss function  $\nabla \mathcal{L}(\theta)$  gives the direction of fastest increase of the loss function  $\mathcal{L}(\theta)$  when the parameters  $\theta$  are changed

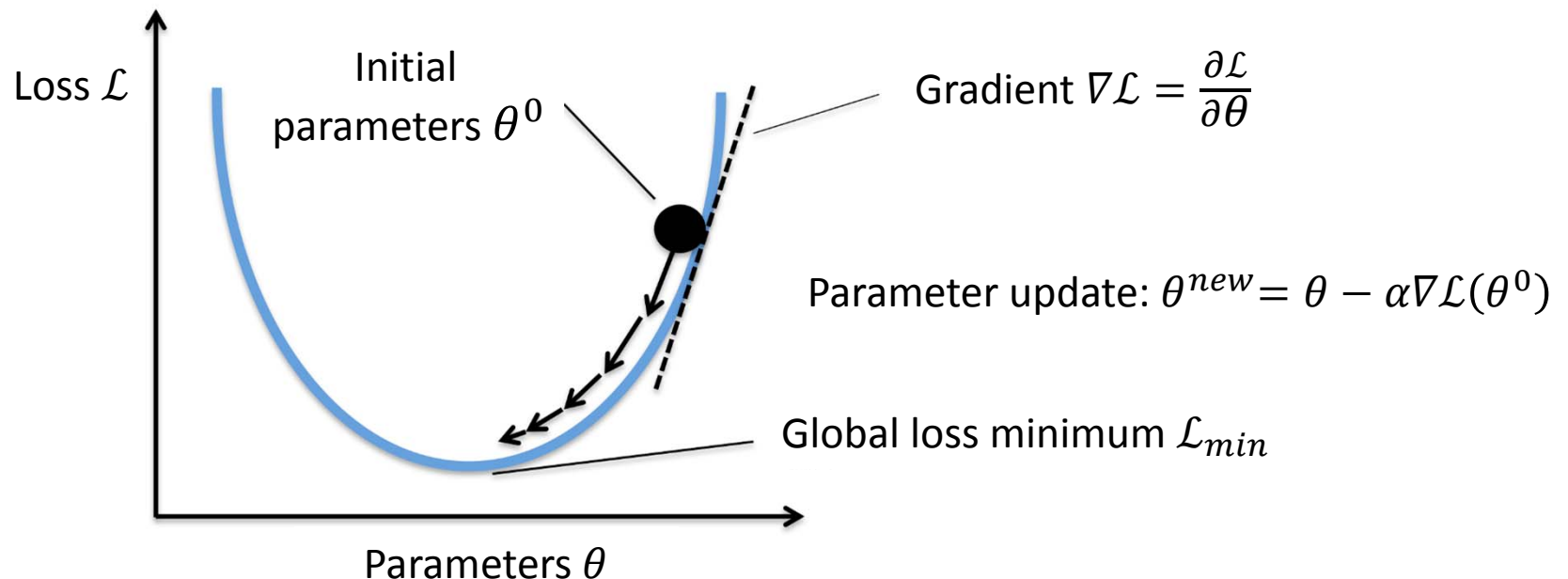


# Gradient Descent Algorithm

*Training Neural Networks*

- Steps in the *gradient descent algorithm*:

1. Randomly initialize the model parameters,  $\theta^0$
2. Compute the gradient of the loss function at the initial parameters  $\theta^0$ :  $\nabla\mathcal{L}(\theta^0)$
3. Update the parameters as:  $\theta^{new} = \theta^0 - \alpha\nabla\mathcal{L}(\theta^0)$ 
  - o Where  $\alpha$  is the learning rate
4. Go to step 2 and repeat (until a terminating criterion is reached)

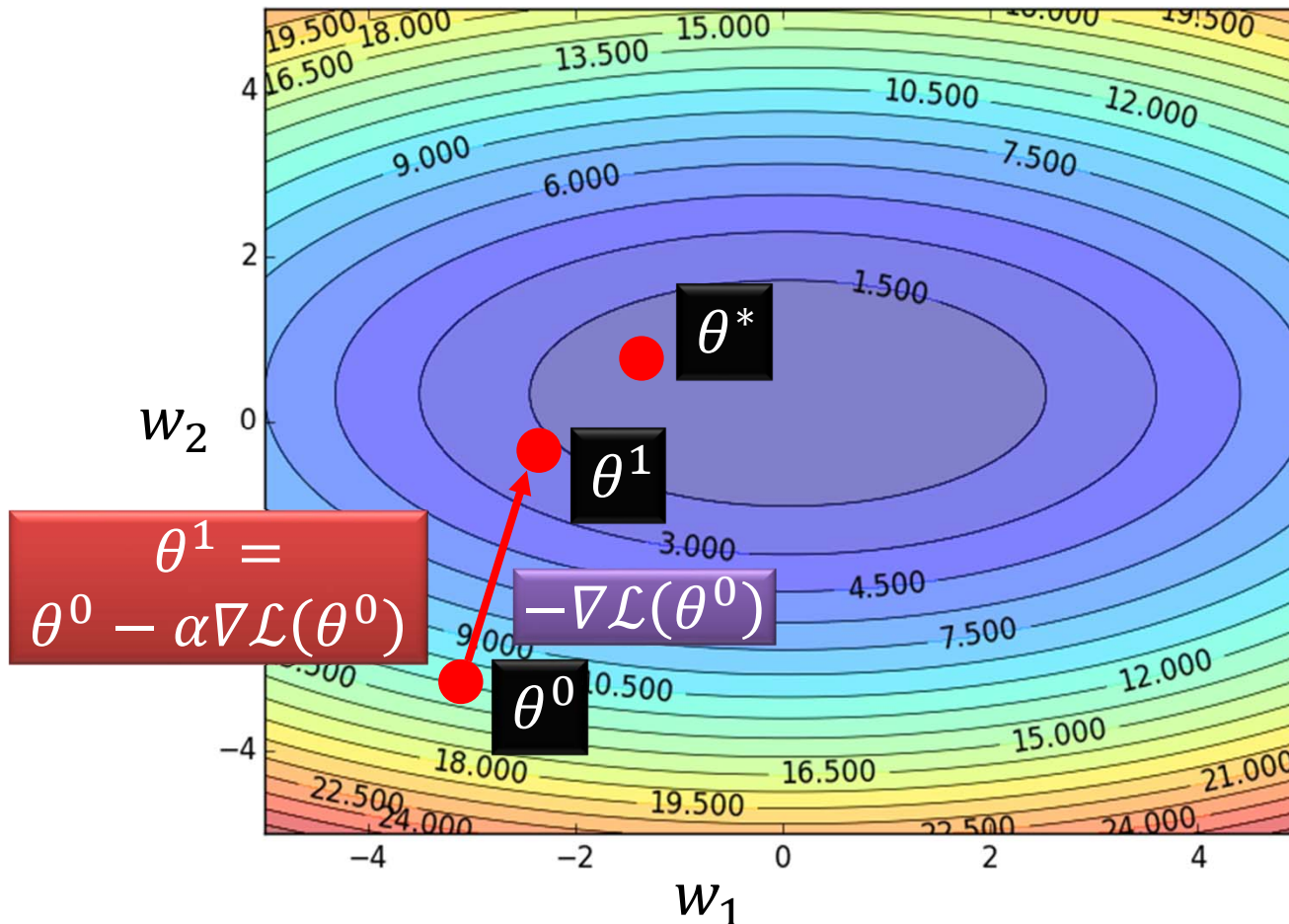




# Gradient Descent Algorithm

## Training Neural Networks

- Example: a NN with only 2 parameters  $w_1$  and  $w_2$ , i.e.,  $\theta = \{w_1, w_2\}$ 
  - The different colors represent the values of the loss (minimum loss  $\theta^*$  is  $\approx 1.3$ )



1. Randomly pick a starting point  $\theta^0$
2. Compute the gradient at  $\theta^0$ ,  $\nabla \mathcal{L}(\theta^0)$
3. Times the learning rate  $\eta$ , and update  $\theta$ ,  
 $\theta^{new} = \theta^0 - \alpha \nabla \mathcal{L}(\theta^0)$
4. Go to step 2, repeat

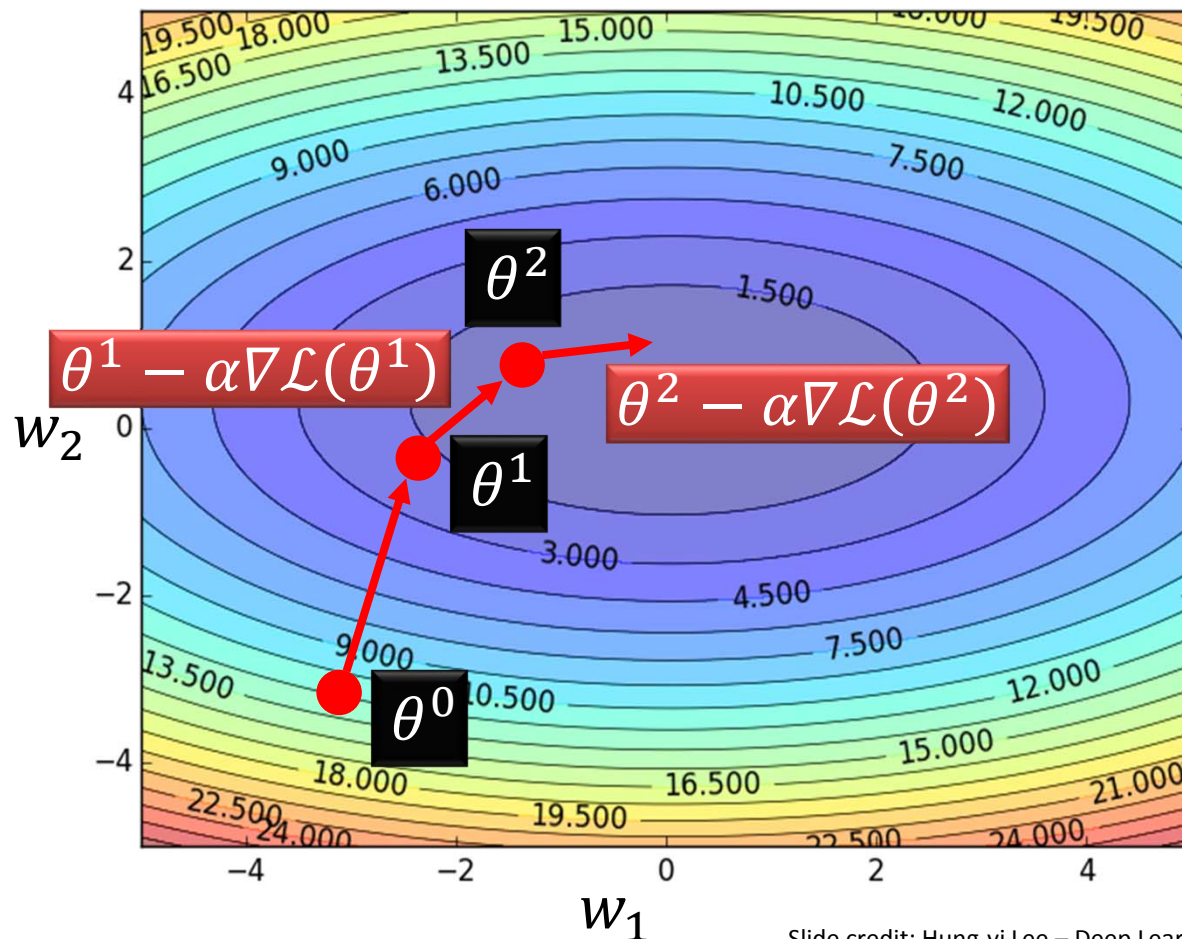
$$\nabla \mathcal{L}(\theta^0) = \begin{bmatrix} \partial \mathcal{L}(\theta^0) / \partial w_1 \\ \partial \mathcal{L}(\theta^0) / \partial w_2 \end{bmatrix}$$

# Gradient Descent Algorithm

Training Neural Networks

- Example (contd.)

Eventually, we would reach a minimum .....



2. Compute the gradient at  $\theta^{old}$ ,  $\nabla \mathcal{L}(\theta^{old})$

3. Times the learning rate  $\eta$ , and update  $\theta$ ,  
 $\theta^{new} = \theta^{old} - \alpha \nabla \mathcal{L}(\theta^{old})$

4. Go to step 2, repeat